

# Architecture Overview

## Tech Stack

- **Frontend:** Next.js 14 (App Router), React, Tailwind CSS, Lucide Icons, Recharts
- **Backend:** Express.js (Node.js API)
- **Database:** PostgreSQL
- **ORM:** Prisma
- **AI/LLM:** Google Gemini API (with local Ollama fallback/support)

## Folder Structure (Monorepo)

The project is structured as a monorepo containing three main workspaces:

- `apps/web/`: The Next.js frontend application.
- `apps/api/`: The Express.js backend API.
- `packages/database/`: The shared Prisma schema, migrations, and seed scripts.

## Authentication Flow

The application uses a **JWT-based authentication flow**.

- Users log in via the `/api/v1/auth/login` endpoint on the Express backend.
- Upon successful authentication, the backend generates a JWT token containing the user's ID and Role.
- The frontend stores this token (in `localStorage/cookies`) and attaches it to the `Authorization: Bearer <token>` header of subsequent API requests via an Axios interceptor.
- Backend routes are protected using the `authenticate` middleware, which verifies the JWT before processing the request.

## Database Schema Summary

Key models in the PostgreSQL database:

- **User:** Represents employees, managers, and admins in the system.
- **Goal:** Core strategic objectives assigned to users, tracking planned vs. actual progress.
- **Cycle:** Time-bound planning periods (e.g., Q2 2026) that group goals and check-ins.
- **CheckIn:** Periodic updates on goal progress, including manager feedback and computed scores.
- **CustomerEmail:** AI-processed support tickets with sentiment, category, and priority classification.
- **CalendarEvent:** Manual reminders and system-generated events (like goal deadlines and check-ins) displayed on the frontend calendar.

## How to Run Locally

Ensure you have Node.js and PostgreSQL installed.

1. **Install dependencies:**

```
npm run install:all
```

## 2. Environment Variables:

Create .env files in apps/api, apps/web, and packages/database matching .env.example.

## 3. Database Setup:

```
npm run db:push  
npm run db:seed
```

## 4. Start Development Servers:

```
npm run dev
```

*This concurrently starts the Express API (port 5000) and Next.js Web (port 3000).*

# Deployment Notes (Vercel)

This monorepo is configured for easy deployment on Vercel:

- **Root vercel.json:** Instructs Vercel to build the apps/web directory and run the npm run build command.
- **Next.js Config:** apps/web/next.config.js is set to output: 'standalone' to optimize the build size.
- **Prisma Generation:** The root package.json includes a postinstall script (npm run db:generate) to ensure the Prisma Client is available immediately during the Vercel build phase, preventing PrismaClientInitializationError.
- **Environment Variables:** Make sure to add DATABASE\_URL, JWT\_SECRET, GEMINI\_API\_KEY, etc., to your Vercel Project Settings.